

A review of

*Optimizing Quantum Compilation
via High-Level Quantum
Instructions*

with explicit tensor reformulations

Juan Giraldo (Universidad Nacional de Colombia)

Introduction to Quantum Computing

June 5, 2026

Original paper: Rosa, Marchi, Duzzioni, de Santiago.

Optimizing Quantum Compilation via High-Level Quantum Instructions. arXiv:2510.24323 (2025).

Contents

Executive Summary	2
1 Background	3
1.1 The compilation pipeline	3
1.2 The with around instruction	3
1.3 What this paper contributes	4
1.4 What this review contributes	5
2 The three theorems and its tensor algebra	6
2.1 Notation conventions	6
2.2 Theorem 1 — Controlled gate reduction	7
2.3 Theorem 2 — Approximate decomposition	10
2.4 Theorem 3 — Safe uncomputation	12
2.5 Summary	15
3 The V-chain example	16
3.1 What the V-chain is	16
3.2 Optimizations in the V-chain	17
3.3 Reproduction	18
3.4 Note on Ket’s compilation choices	19
Conclusion	22

Executive Summary

Rosa, Marchi, Duzzioni and de Santiago (2025) [10] prove three theorems that license quantum compiler optimizations triggered by the `with around` high-level instruction in the Ket platform [4]. The theorems show that:

1. Outer controls wrapping an $A^\dagger BA$ segment can be placed only on B , leaving A and $A^\dagger$ uncontrolled.
2. Inside a $P^\dagger D \cdot C^n U \cdot P$ segment with appropriate structure on D , the permutation gate P can be replaced by a cheaper approximate version $\tilde{P}$ without changing the segment's final unitary.
3. Auxiliary qubits initialized to $|0\rangle$ inside an $A^\dagger BA$ segment of the right form are guaranteed to return to $|0\rangle$.

Applied to the V-chain decomposition [1] of the multi-controlled NOT gate, the optimizations together reduce the CNOT count from 54 to 30 for $n = 6$ control qubits, approaching a 50% reduction asymptotically. This review reformulates the three theorems with the tensor structure of all operators made explicit. The reformulations in Chapter 2 write each operator with all its register factors visible, name each register's role, and state each hypothesis in terms that improves clarity on register membership. The review also reproduces the paper's headline empirical claim on a local Ket installation. The reproduction confirms the 30 vs 54 CNOT result for $n = 6$ and extends the comparison to n up to 19, showing the reduction approaching 50% as predicted by the paper. The full reproduction script and CNOT-count table are in Chapter 3. **Outline.** Chapter 1 introduces the quantum compilation pipeline and

the `with around` instruction. Chapter 2 contains the tensor-explicit reformulations of the three theorems. Chapter 3 applies them to the V-chain example and documents the local reproduction.

1 Background

1.1 The compilation pipeline

A quantum program written in a high-level platform like Qiskit [6], Cirq, or Ket [4] cannot run directly on hardware. Real quantum processors execute only a small set of native operations — the ISA — determined by the physics of the device. Everything else must be lowered to that ISA before execution.

The lowering happens in three stages. Multi-qubit gates are decomposed into sequences of 1- and 2-qubit gates. Logical qubits are then mapped onto physical qubits, respecting the hardware’s connectivity graph [7]. Finally, the 1- and 2-qubit gates are translated into the device’s native gate set (typically something like $\{R_Z, \sqrt{X}, X, \text{CNOT}\}$ for superconducting platforms).

The first stage — multi-qubit gate decomposition — is where this paper lives. The choice of decomposition algorithm directly determines the gate count of the final circuit, which in turn determines the total error and execution time. On current NISQ hardware, CNOT gates have roughly $10\times$ the error rate of single-qubit gates, so the CNOT count is the dominant cost metric for any decomposition strategy.

1.2 The with around instruction

The pattern $A^\dagger \cdot B \cdot A$ — apply a gate, do something in the conjugated frame, then undo the gate — appears constantly in quantum algorithms. The Ket platform exposes this pattern as a high-level language construct [9]:

```
with around(A, qubits):  
    # body B applied here  
# A^dagger applied automatically at end of block
```

Equivalent code without the instruction would write `A`, then `B`, then `A.adj()` as three separate statements. The two are semantically identical, but they are not equivalent from the compiler’s perspective: the `with around` form tells the compiler that the segment has the $A^\dagger B A$ structure, while the explicit form looks like three independent gate calls. This structural information is the foundation for the three theorems we will examine in Chapter 2.

The shift in framing here mirrors the classical computing transition from assembly to high-level languages. In classical code, writing a `for` loop instead of manual `jump` statements does not change the program’s semantics, but it gives the compiler enough structural information to apply loop unrolling, vectorization, and parallelization. Quantum programming is currently dominated by circuit-level constructs (gate placement, qubit indexing) that correspond to the assembly stage. `with around` is one of the high-level constructs that exposes algebraic structure to the compiler in a way that unlocks specifically quantum optimizations.

1.3 What this paper contributes

The paper [10] proves three theorems about circuit segments that have the $A^\dagger B A$ shape:

- **Theorem 1** (controlled gate reduction) lets the compiler drop outer controls on A and $A^\dagger$ when they appear inside a larger controlled operation, putting the controls only on the body B .
- **Theorem 2** (approximate decomposition) lets the compiler substitute a cheaper approximate version $\tilde{P}$ for a permutation gate P inside the segment, provided D is diagonal on the right register. The approximate Toffoli of Maslov [8] is the canonical example.
- **Theorem 3** (safe uncomputation) guarantees that auxiliary qubits initialized to $|0\rangle$ inside a `with around` block return to $|0\rangle$ at the end of the block, separable from the rest of the system. This formalizes a notion of safe uncomputation previously studied in language-level form by Silq [3], building on the original concept of reversible computation [2].

Each theorem licenses a specific compiler optimization. Together they let the compiler apply transformations to user-written code that would otherwise require manual verification by the programmer.

The paper demonstrates the combined effect of all three theorems through a V-chain decomposition [1] of the multi-controlled NOT gate, showing a reduction from 54 CNOTs to 30 CNOTs for $n = 6$ controls — a 44% reduction approaching a 50% asymptotic limit as n grows.

1.4 What this review contributes

The paper’s theorems are correct and the proofs are valid, but the presentation makes algebraic moves that depend on the tensor structure of the operators involved without writing that structure explicitly. Several conditions in the hypotheses — particularly the requirement that D be diagonal *on a specific register* — are stated in prose rather than in the formal theorem statements, leaving the reader to reconstruct the tensor decomposition from context.

This review reformulates the three theorems with the tensor structure made explicit. Each operator is written with its $\otimes I$ factors visible, each register’s role is named, and each hypothesis is stated in terms that clarify register membership. The proofs in the original paper still hold; the reformulations only change what the reader must reconstruct to verify them.

Chapter 2 contains the reformulations. Chapter 3 applies them to the V-chain example and reproduces the paper’s claim on a local ket installation.

2 The three theorems and its tensor algebra

The central technical content of Rosa et al. [10] is three theorems that justify three compiler optimizations. Each theorem describes a circuit segment, states conditions on the operators inside it, and concludes that some transformation of that segment preserves the final unitary or returns auxiliary qubits to a clean state.

The theorems are correct as stated, but the paper’s presentation makes algebraic moves that depend on *which qubits each operator touches* without writing those tensor factors explicitly. As a reader trying to verify the proofs, the most common stumbling block is reconstructing the implicit $\otimes I$ factors that distinguish, for instance, “ D is diagonal” (on the full joint system) from “ D is diagonal on the P-register” (which is what the proof actually requires). The two are not equivalent.

This chapter restates the three theorems with the tensor structure made visible. For each theorem, we summarize the paper’s presentation, point out what is left implicit, and then give a reformulation that exposes the registers and the $\otimes I$ factors explicitly. We do not redo the proofs but we make the hypotheses precise enough that the reader can follow the original proofs without having to reconstruct the implicit tensor structure.

2.1 Notation conventions

Throughout this chapter we work with quantum circuits acting on a joint system composed of two disjoint qubit registers:

- The **P-register** — the qubits that the permutation gate P acts on. We will see that in Theorems 2 and 3 this register also plays the role of the control register for the inner controlled operation $C^n U$.

- The **U-register** — the qubits that the inner unitary U acts on. This register is disjoint from the P-register; no gate in the segment ever directly modifies its computational basis except U itself.

The full joint Hilbert space is the tensor product of the two:

$$\mathcal{H} = \mathcal{H}_{\text{P-reg}} \otimes \mathcal{H}_{\text{U-reg}}.$$

Every operator in the circuit is a unitary on this joint space, but in general acts non-trivially on only one register. We adopt the convention that an operator written without an explicit tensor factor acts as identity on the unspecified register. So P means $\hat{P} \otimes I_{\text{U-reg}}$, and U inside $C^n U$ means $I_{\text{P-reg}} \otimes \hat{U}$, where the hat distinguishes the register-local action from the full-system operator.

Projectors on the P-register

For the controlled operations in Theorems 2 and 3 we use the following projectors on $\mathcal{H}_{\text{P-reg}}$, where n is the number of qubits in the P-register:

$$P_{\neq}^{\text{P}} := \sum_{k=0}^{2^n-2} |k\rangle\langle k| \quad (\text{“not all controls are } |1\rangle\text{”})$$

$$P_1^{\text{P}} := |2^n - 1\rangle\langle 2^n - 1| \quad (\text{“all controls are } |1\rangle\text{”})$$

These are orthogonal complements: $P_{\neq}^{\text{P}} P_1^{\text{P}} = 0$ and $P_{\neq}^{\text{P}} + P_1^{\text{P}} = I_{\text{P-reg}}$. They partition the P-register’s basis states into the all-ones state and everything else — which is exactly the partition that the controlled gate $C^n U$ uses to decide whether U fires.

With this notation, the controlled gate is

$$C^n U = P_{\neq}^{\text{P}} \otimes I_{\text{U-reg}} + P_1^{\text{P}} \otimes \hat{U}.$$

The first term is the identity on the U-register when the P-register is not all-ones; the second term fires $\hat{U}$ on the U-register when the P-register is all-ones. Both terms preserve the P-register’s basis state — the projectors only check it, they do not modify it.

2.2 Theorem 1 — Controlled gate reduction

Paper statement

Theorem 1 of Rosa et al. [10] states the identity

$$C^n(A^\dagger B A) = A^\dagger \cdot C^n B \cdot A.$$

The paper presents this as an optimization for circuits where outer controls wrap a **with around** segment: rather than building controlled versions of A and $A^\dagger$, the compiler can drop the controls on those gates and put them only on B . The proof in the paper proceeds by case analysis on whether all n control qubits are $|1\rangle$.

What is left implicit

The identity above has A on one side and $C^n(\cdot)$ on the other. For the equation to make sense as an operator equation, A and $C^n B$ must act on the same joint Hilbert space. This requires reading A as extended by identity on the control register: $A = I_{\text{ctrl}} \otimes \hat{A}_{\text{target}}$.

Once that extension is made explicit, the proof becomes a one-line algebraic statement rather than a case analysis: A acts on a register that is disjoint from the register where the control projectors live, so A factors through the projector sum cleanly.

Reformulation

Let the joint system be $\mathcal{H}_{\text{ctrl}} \otimes \mathcal{H}_{\text{target}}$ with n qubits in the control register. Let A and B be unitaries on $\mathcal{H}_{\text{target}}$.

Theorem 1, reformulated

With operators on the joint system written as

$$\begin{aligned} A &= I_{\text{ctrl}} \otimes \hat{A}_{\text{target}}, \\ B &= I_{\text{ctrl}} \otimes \hat{B}_{\text{target}}, \\ C^n B &= P_{\neq} \otimes I_{\text{target}} + P_1 \otimes \hat{B}_{\text{target}}, \\ C^n(A^\dagger B A) &= P_{\neq} \otimes I_{\text{target}} + P_1 \otimes \hat{A}_{\text{target}}^\dagger \hat{B}_{\text{target}} \hat{A}_{\text{target}}, \end{aligned}$$

the identity

$$C^n(A^\dagger B A) = A^\dagger \cdot C^n B \cdot A$$

holds, and is a direct consequence of three structural facts:

1. Operators on disjoint tensor factors commute as
 $(X \otimes I)(I \otimes Y) = (I \otimes Y)(X \otimes I)$
2. Composition of operators on disjoint factors satisfies
 $(I \otimes \hat{A}^\dagger) \cdot (X \otimes Y) \cdot (I \otimes \hat{A}) = X \otimes (\hat{A}^\dagger Y \hat{A})$
3. Unitarity: $\hat{A}^\dagger \hat{A} = I$

The first two facts let $A^\dagger$ and A pass through the projector sum on the control register without seeing it; the third collapses the $P_{\neq}$ term to identity on the target.

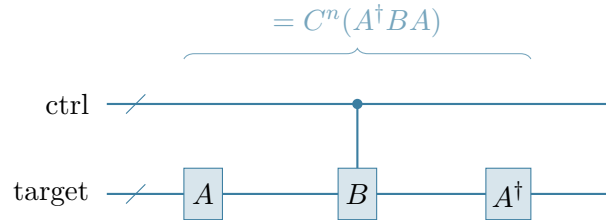


Figure 2.1: Theorem 1 reformulated. The outer control on the segment applies only to B — the gates A and $A^\dagger$ act on the target register only and are extended by identity on the control register, so they pass through the projector sum on the control register without picking up controls.

The point of the reformulation is that the identity is purely structural — the

cancellation $A^\dagger \cdot I \cdot A = I$ happens on every term of the sum that does not contain B , and the only term that contains B gets $A^\dagger B A$ assembled inside it. No case analysis is required once the tensor product is written explicitly.

2.3 Theorem 2 — Approximate decomposition

Paper statement

Theorem 2 considers a circuit segment of the form $P^\dagger D P$ where P is a permutation gate, D is a diagonal gate, and (in the full circuit) there is also a controlled operation $C^n U$ inside the segment. The theorem says that if $\tilde{P} = D_p P$ is an approximate version of P differing only by a diagonal phase D_p , then substituting $\tilde{P}$ for P everywhere in the segment produces the same final unitary — provided D preserves the computational basis of the qubits P acts on.

This is the optimization that lets the compiler replace an exact Toffoli (6 CNOTs) with an approximate Toffoli [8] (3 CNOTs) inside a `with around` block.

What is left implicit

The theorem statement says “ $P^\dagger D P$ ” but the circuit shown in Eq. 9 of the paper is $P^\dagger D \cdot C^n U \cdot P$, with a controlled operation sitting between D and the final P . The controlled gate is essential to the optimization’s value — if there were nothing between P and $P^\dagger$, the segment would be a no-op and the substitution would be trivial. But its presence is announced inside the theorem’s clarifying paragraph rather than in the statement itself.

More substantively, the condition “ D is a diagonal gate” is not specific enough. A gate that is diagonal on the full joint system can still attach phases that depend on the U-register, and those phases would survive the $P/P^\dagger$ cancellation and break the substitution argument. The condition the proof actually requires is that D is diagonal *on the P-register specifically* — the phases attached by D may depend on basis states of the P-register but not on basis states of the U-register. The paper expresses this later in prose as “ D preserves the computational basis of the qubits P acts on,” but the formal hypothesis would benefit from being written in tensor form from the start.

A third subtlety: the same register that P acts on must serve as the *control register* of the inner $C^n U$. This dual role is what couples the inner sandwich together.

Reformulation

Theorem 2, reformulated

Let the joint system be $\mathcal{H}_{\text{P-reg}} \otimes \mathcal{H}_{\text{U-reg}}$ and consider the circuit segment

$$\mathcal{S} := P^\dagger D C^n U P$$

acting on this joint system, where:

(H1) P is a permutation gate confined to the P-register:

$$P = \left(\sum_k e^{i\phi_k} |p(k)\rangle\langle k| \right) \otimes I_{\text{U-reg}}.$$

(H2) D is diagonal on the P-register:

$$D = \left(\sum_k e^{i\theta_k} |k\rangle\langle k| \right) \otimes I_{\text{U-reg}}.$$

The phases $e^{i\theta_k}$ depend only on basis states of the P-register.

(H3) $C^n U$ uses the P-register as its control register and the U-register as its target register:

$$C^n U = P_{\neq}^{\text{P}} \otimes I_{\text{U-reg}} + P_1^{\text{P}} \otimes \hat{U}_{\text{U-reg}}.$$

The unitary $\hat{U}$ is arbitrary.

(H4) $\tilde{P} = D_p P$ where D_p is diagonal on the P-register (same structural form as D in H2).

Then

$$\tilde{P}^\dagger \cdot D \cdot C^n U \cdot \tilde{P} = P^\dagger \cdot D \cdot C^n U \cdot P,$$

that is, the substitution $P \mapsto \tilde{P}$ leaves the segment's unitary unchanged.

Now we see:

- All four operators in the segment are extended with explicit $\otimes I$ factors, so it is unambiguous which register each one acts on.

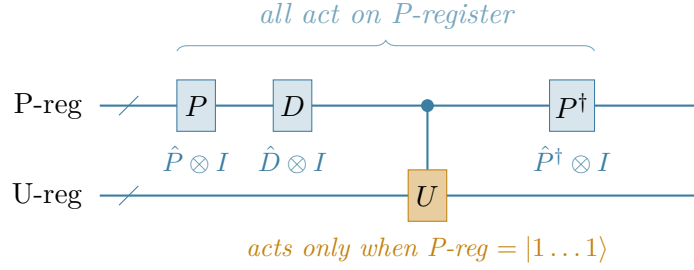


Figure 2.2: Theorem 2 reformulated. The permutation P , the diagonal D , and $P^\dagger$ all act on the P-register only (extended by identity on the U-register). The controlled gate $C^n U$ uses the P-register as its control and the U-register as its target. The substitution $P \mapsto \tilde{P}$ is safe because $\tilde{P}$ differs from P only by a diagonal phase on the P-register, which cancels against $\tilde{P}^\dagger$ at the end of the segment.

- H2 makes the diagonal-on-which-register condition part of the hypothesis. Without H2 in this form, an operator diagonal on the full joint system but coupling the two registers through its phases could break the cancellation.
- H3 makes the dual role of the P-register explicit: it is simultaneously the target of P (from H1) and the control of $C^n U$ (from H3).
- The cancellation mechanism that makes the substitution safe is purely structural: D_p commutes with D and with $C^n U$ because all three are built from operators diagonal on the P-register, and $D_p^\dagger D_p = I$ at the end of the segment cancels the phase error that D_p introduced at the start.

2.4 Theorem 3 — Safe uncomputation

Paper statement

Theorem 3 considers a circuit segment $A^\dagger B A$ where A acts on an auxiliary register α initialized to $|0 \dots 0\rangle$. The theorem says that after the segment closes, the auxiliary register returns to $|0 \dots 0\rangle$ separable from the rest of the system — meaning it can be safely freed and reused without leaving residual entanglement [2, 3].

This is the result that lets the `@using_aux` decorator dynamically allocate auxiliary qubits and release them at the end of a `with around` block.

What is left implicit

The theorem's proof in the paper writes the operator $A^\dagger BA$ as a three-term sum (Eq. 22) without showing how those three terms were obtained. Reading the proof carefully, the three-term sum is not the bare operator: it is the operator after applying the projector $|0\rangle\langle 0|_\alpha$ to extract the action on a clean ancilla. The bare operator $A^\dagger BA$ has four terms, of which one vanishes under the clean-ancilla projection.

This is a presentational choice rather than an error but the reader cannot recover the algebra without first guessing what the projection has done. As a teaching artifact, it is cleaner to compute the bare operator, then apply the clean-ancilla projection as an explicit step.

The same tensor-structural points from Theorem 2 reappear here: A is a controlled-permutation acting on auxiliary qubits while B couples those qubits as controls to a separate register; the auxiliary register plays a dual role; the diagonal D 's phases must depend only on the auxiliary qubits.

Reformulation

Theorem 3, reformulated

Let the joint system have three disjoint registers:

- ψ — the outer control register
- ϕ — the target register
- α — the auxiliary register, with $a \geq 1$ qubits

Consider the circuit segment

$$\mathcal{S} := A^\dagger B A$$

on the joint space $\mathcal{H}_\psi \otimes \mathcal{H}_\phi \otimes \mathcal{H}_\alpha$, where:

(H1) A is a controlled-permutation gate:

$$A = \left(P_{\neq}^\psi \otimes I_\alpha + P_{\mathbf{1}}^\psi \otimes \hat{P}_\alpha \right) \otimes I_\phi,$$

that is, A applies the permutation $\hat{P}_\alpha$ to the auxiliary register when the outer control ψ is all-ones, and acts as identity otherwise.

(H2) B has the form $D \cdot C^a U$ where D is diagonal on the auxiliary register and $C^a U$ uses the auxiliary register as control and ϕ as target:

$$B = I_\psi \otimes I_\phi \otimes \hat{D}_\alpha + I_\psi \otimes \hat{U}_\phi \otimes \hat{D}_{\mathbf{1},\alpha},$$

where $\hat{D}_{\mathbf{1},\alpha}$ is the restriction of $\hat{D}_\alpha$ to the all-ones subspace of the auxiliary register, and $\hat{D}_\alpha$ on the not-all-ones subspace is paired with I_ϕ .

(H3) The auxiliary register is initialized to $|0 \dots 0\rangle$.

Then the operator $\mathcal{S} \cdot (I_{\psi\phi} \otimes |0 \dots 0\rangle\langle 0 \dots 0|_\alpha)$ factorizes as

$$\mathcal{S}|_{\alpha=|0\rangle} = (\text{some unitary on } \psi \otimes \phi) \otimes e^{i\theta} |0 \dots 0\rangle\langle 0 \dots 0|_\alpha,$$

where $e^{i\theta}$ is a global phase. The auxiliary register exits in $|0 \dots 0\rangle$, separable from $\psi \otimes \phi$, and can be safely freed.

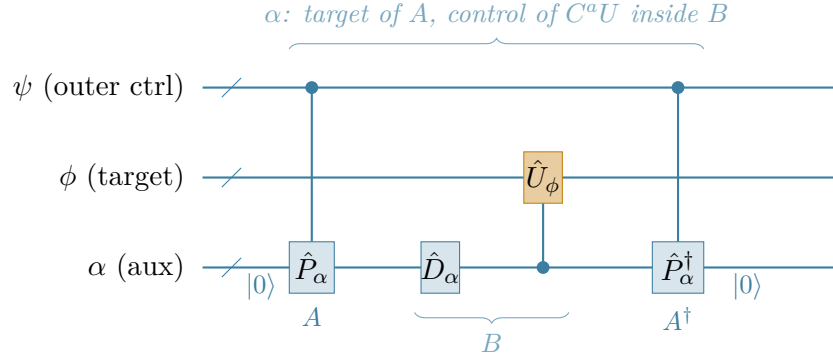


Figure 2.3: Theorem 3 reformulated. Three disjoint registers: ψ (outer control), ϕ (target), α (auxiliary, initialized to $|0\rangle$). A applies $\hat{P}_\alpha$ to the auxiliary register conditional on ψ being all-ones; inside B , the same auxiliary register acts as the control of $\hat{U}_\phi$. After $A^\dagger$ closes the segment, α returns to $|0\rangle$ separable from $\psi \otimes \phi$.

Now we see:

- All operators are written with explicit factors over the three registers ψ , ϕ , α . The auxiliary register’s membership and role are unambiguous.
- The clean-ancilla hypothesis (H3) is stated as a property of the initial state, separate from the operator $A^\dagger B A$ itself.
- The factorization in the conclusion makes explicit that the auxiliary register exits in a pure basis state $|0 \dots 0\rangle$ multiplied by a scalar phase (“global phase”)

2.5 Summary

The three theorems share a common pattern: a circuit segment whose structure depends on the membership of operators in specific registers, together with hypotheses on how those operators are constrained inside those registers. The original paper proves the theorems correctly, but its presentation does not make register membership part of the formal statement, leaving the reader to reconstruct the tensor structure from context.

The reformulations above make register membership explicit by writing every operator with all its $\otimes I$ factors.

The next chapter applies these reformulations to the V-chain example that the paper uses to demonstrate its results, and reproduces the paper’s headline CNOT-count reduction on a local Ket installation.

3 The V-chain example

The paper demonstrates its three theorems through an implementation of the V-chain decomposition [1] for the multi-controlled NOT gate. The same algorithm is implemented twice — once using the `with around` instruction and once using explicit gate calls — and the two implementations are compiled to compare their CNOT counts. This chapter reviews the example using the tensor-explicit framing from Chapter 2, and reproduces the paper’s headline result on a local Ket installation [4].

3.1 What the V-chain is

The V-chain [1] is a recursive decomposition of the n -controlled NOT gate into a sequence of Toffoli (3-qubit controlled-controlled-NOT) gates plus a single base-case Toffoli, using $n - 2$ auxiliary qubits. Each Toffoli computes the AND of two controls into an auxiliary qubit. The chain of ANDs accumulates the multi-way AND condition one step at a time; the final auxiliary qubit serves as the single control of the flip on the target. After the target has been flipped (or left alone, depending on the input), the Toffolis run in reverse to return the auxiliary qubits to $|0\rangle$. This second phase is uncomputation: it has no operational effect on the answer but is required to keep the auxiliary qubits disentangled and safe to free. The cost trade is gates versus qubits. Without auxiliary qubits, the best known exact decomposition of $C^n X$ uses $O(n^2)$ CNOTs [5]. With $n - 2$ auxiliary qubits available, the V-chain reduces this to $O(n)$ CNOTs. The optimization in the paper improves the V-chain further by replacing the exact Toffoli (6 CNOTs each) with the approximate Toffoli [8] (3 CNOTs each) at every recursion level except the base case.

3.2 Optimizations in the V-chain

The V-chain's recursive structure produces nested **with around** blocks. Each recursive step has the form

```
with around(Toffoli, aux):  v_chain(aux + remaining_controls, target)
```

which expands to a circuit segment $T^\dagger \cdot (\text{recursive body}) \cdot T$, with T a Toffoli permuting basis states on the auxiliary qubit. This is the canonical Theorem 2 shape with:

- P-register = the two original controls plus the auxiliary qubit
- U-register = the target qubit
- $P = T$ (a permutation gate by H1 of Theorem 2)
- The inner body is a recursive call that uses the auxiliary qubit *only as a control*, contributing diagonal-on-P-register phases (H2) and a $C^n U$ structure with the auxiliary qubit as part of the control (H3)

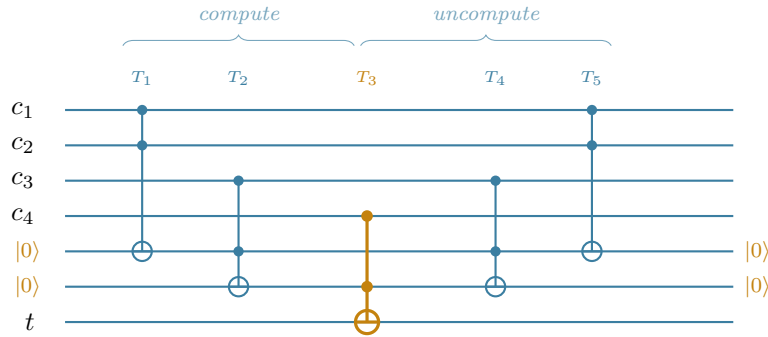


Figure 3.1: V-chain for $C^4 X$ using 2 auxiliary qubits. Each Toffoli computes the running AND of one more control into the next auxiliary qubit. The base-case Toffoli T_3 (in orange) is the only gate that actually flips the target t . The Toffolis T_4 and T_5 run the chain in reverse to return the auxiliary qubits to $|0\rangle$. T_1, T_2, T_4, T_5 sit inside matched **with around** pairs and qualify for the approximate Toffoli substitution from Theorem 2; T_3 stays exact because it has no matched adjoint inside any **with around** block.

Theorem 2 then licenses replacing T with the approximate Toffoli $\tilde{T}$ at this layer. Theorem 3 guarantees that the auxiliary qubit returns to $|0\rangle$ at the end of the layer,

since it satisfies H1–H3 of the reformulation in Section 2.4. Theorem 1 does not fire in the standalone case, but would activate the moment a larger circuit wraps the V-chain in additional outer controls. The recursive composition makes both theorems fire at every recursion level except the base case. The base-case Toffoli T_{base} performs the actual flip on the target and is not inside any matched adjoint pair, so Theorem 2 does not apply to it and it stays at 6 CNOTs.

3.3 Reproduction

To verify the paper’s empirical claim, we ran both implementations on a local Ket 0.9.3 [4] installation under Python 3.14 on macOS. The script allocates the required qubits, calls one of the two implementations, configures an all-to-all coupling graph to force the compilation pipeline to lower multi-qubit gates, and reads the CNOT count from the compiled circuit’s logical gate metadata.

Two implementations

The implementation using `with around` is the same code shown in the paper’s Figure 3 [10]:

```
@using_aux(a=lambda c: int(len(c) > 2))
def v_chain(c, t, a):
    if len(c) <= 2:
        ctrl(c, X)(t)
    else:
        with around(ctrl(c[:2], X), a):
            v_chain(a + c[2:], t)
```

The explicit version, also from Figure 3 of [10], writes the uncomputation step manually and threads the auxiliary qubits through the recursion:

```
def v_chain_x(c, t, a):
    if len(c) <= 2:
        ctrl(c, X)(t)
    else:
        ctrl(c[:2], X)(a[0])
        v_chain_x(a[0] + c[2:], t, a[1:])
        ctrl(c[:2], X)(a[0])
```

The two functions compute the same logical operation. The difference is that the first exposes the $A^\dagger B A$ structure to the compiler, which triggers Theorems 2 and 3; the second produces three independent gate calls that the compiler cannot recognize as a matched adjoint pair.

Results

Table 3.1 shows the CNOT count for each implementation across a range of control counts, measured directly from Ket’s compilation metadata. The headline result for

n	Optimized	Explicit	Reduction
3	14	18	22.2%
4	18	30	40.0%
5	24	42	42.9%
6	30	54	44.4%
7	36	66	45.5%
8	42	78	46.2%
9	48	90	46.7%
10	54	102	47.1%
12	66	126	47.6%
14	78	150	48.0%
16	90	174	48.3%
18	102	198	48.5%
19	108	210	48.6%

Table 3.1: CNOT count for the optimized and explicit implementations of $C^n X$ as n varies. The $n = 6$ row matches the paper’s headline result of 30 vs 54 CNOTs.

$n = 6$ matches the paper exactly: 30 CNOTs in the optimized version versus 54 CNOTs in the explicit version, a 44.4% reduction. The reduction grows monotonically with n and approaches 50% asymptotically, matching the paper’s claim about asymptotic behavior.

3.4 Note on Ket’s compilation choices

The optimized count of 30 CNOTs for $n = 6$ is reached when the compiler applies the `NoAuxCX` algorithm with approximate Toffoli substitution, which is what the `V-chain` function with `with around` triggers. If the programmer skips the `v_chain` helper entirely and just calls `ctrl(c, X)(t)` directly, Ket’s compiler chooses a different

algorithm (`NetworkPauli(C2X)`) that also reaches 30 CNOTs by a different route. The paper’s footnote [10] acknowledges this: “Ket’s compiler can automatically decompose multi-controlled gates.” Practically, this means the `v_chain` function in the paper is an *illustrative* demonstration of how a programmer could write the V-chain by hand and have the compiler apply the optimizations to that handwritten code. The compiler’s built-in routine for multi-controlled gates already does the equivalent work without the helper. The contribution of the paper is not that the V-chain function is the fastest way to build C^nX in Ket — it is that the `with around` instruction makes the compiler’s optimizations applicable to *user-written* circuits, not just to built-in operations. The reproduction confirms this. The explicit version, which does not use `with around`, produces 54 CNOTs at $n = 6$. Whether the optimized version is reached through the V-chain helper or through the compiler’s built-in algorithm, the result is the same: 30 CNOTs.

Conclusion

The paper’s three theorems [10] and their accompanying **with around** instruction make a small but real contribution to quantum compilation: they show that high-level constructs in quantum programming languages can give compilers enough algebraic structure to apply transformations that the compiler could not safely apply to raw gate sequences. The V-chain example [1] demonstrates this concretely with a 44% CNOT reduction at $n = 6$ that we reproduced locally. The reformulations in Chapter 2 do not change the mathematics of the theorems — they only make the tensor structure of the operators visible at the level of the theorem statements. The hypotheses become easier to verify because each register’s role is explicit; the proofs become easier to follow because the algebraic moves no longer depend on implicit identity factors. The contribution of this review is purely expository: a clearer presentation of results that were already correct. Several open directions remain in the paper itself. Theorem 3 covers clean auxiliary qubits (those initialized to $|0\rangle$), but the analogous result for dirty auxiliary qubits (those in unknown initial states) is not addressed. The static analysis required to identify **with around** blocks that satisfy the theorems’ hypotheses is described informally; a formal account of how the compiler should implement this check would be a natural next step. Finally, the optimizations are stated at the gate-decomposition stage of compilation; the qubit-mapping stage [7], where physical connectivity constraints come into play, is left for future work. These open questions notwithstanding, the paper’s central claim — that high-level instructions can be the right abstraction layer for quantum compiler optimization — is supported by both the theorems and the V-chain demonstration. The parallel to classical compilation, where high-level language constructs eventually surpassed handwritten assembly, is a useful framing for where quantum compilation may be heading.

Bibliography

- [1] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter.
Elementary gates for quantum computation.
Physical Review A, 52(5):3457–3467, 1995.
doi: 10.1103/PhysRevA.52.3457.
- [2] Charles H. Bennett.
Logical reversibility of computation.
IBM Journal of Research and Development, 17(6):525–532, 1973.
doi: 10.1147/rd.176.0525.
- [3] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev.
Silq: a high-level quantum language with safe uncomputation and intuitive semantics.
In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 286–300, 2020.
doi: 10.1145/3385412.3386007.
- [4] Evandro C. R. Da Rosa and Rafael De Santiago.
Ket quantum programming.
ACM Journal on Emerging Technologies in Computing Systems, 18(1):1–25, 2022.
doi: 10.1145/3474224.
- [5] Adenilton J. Da Silva and Daniel K. Park.
Linear-depth quantum circuits for multiqubit controlled gates.
Physical Review A, 106(4):042602, 2022.
doi: 10.1103/PhysRevA.106.042602.

- [6] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, et al.
Quantum computing with Qiskit.
arXiv preprint arXiv:2405.08810, 2024.
doi: 10.48550/arXiv.2405.08810.
- [7] Gushu Li, Yufei Ding, and Yuan Xie.
Tackling the qubit mapping problem for NISQ-era quantum devices.
In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 1001–1014, 2019.
doi: 10.1145/3297858.3304023.
- [8] Dmitri Maslov.
Advantages of using relative-phase Toffoli gates with an application to multiple control Toffoli optimization.
Physical Review A, 93(2):022311, 2016.
doi: 10.1103/PhysRevA.93.022311.
- [9] Evandro C. R. Rosa, Eduardo I. Duzzioni, and Rafael De Santiago.
Optimizing gate decomposition for high-level quantum programming.
Quantum, 9:1659, 2025.
doi: 10.22331/q-2025-03-12-1659.
- [10] Evandro C. R. Rosa, Jerusa Marchi, Eduardo I. Duzzioni, and Rafael de Santiago.
Optimizing quantum compilation via high-level quantum instructions.
arXiv preprint arXiv:2510.24323, 2025.
doi: 10.48550/arXiv.2510.24323.